IMovies – Movie Ticket Booking System

Project Design Phase

Architecture, data design and UML-ready specifications

Project documentation based on the uploaded bookMyShow-main source ZIP

Prepared: 31 October 2026

Project Design Phase

3.1 Architecture

Three logical layers: (1) React client with pages, Redux state and Axios API modules; (2) Express server with route modules and JWT middleware; (3) MongoDB persistence through Mongoose models. The browser calls the backend over HTTP.

3.2 Architecture flow

Layer	Responsibilities	Source locations
Presentation	Registration, login, home/movie pages, booking, profile and admin UI	client/src/pages, App.js
API / business logic	REST endpoints, auth checks, booking/show/movie/theatre operations	server/index.js, server/routes
Data	Users, movies, theatres, shows and bookings	server/models, config/dbConfig.js

3.3 Data entities

Entity	Important fields / relationships
User	name, email (unique), password hash, isAdmin
Movie	title, description, genre, language, poster, releaseDate
Theatre	name, email, address, phone, isActive, owner reference
Show	name, date, time, ticketPrice, totalSeats, bookedSeats, movie reference, theatre reference
Booking	show reference, user reference, seats array, transactionId, timestamps

3.4 ER diagram description

Draw five entities: USER, MOVIE, THEATRE, SHOW and BOOKING. A MOVIE can be associated with many SHOW records; a THEATRE can host many SHOW records; a USER can create many BOOKINGS; each BOOKING refers to one SHOW. A SHOW maintains its bookedSeats array. Use one-to-many connectors from Movie to Show, Theatre to Show, User to Booking and Show to Booking.

3.5 Use-case diagram description

Actors: Visitor, Registered User and Administrator. Visitor use cases: browse movies, register, log in. Registered User: view movie details, view theatres/shows, select seats, book tickets, view booking history. Administrator: add/edit/delete movies, manage theatres and create/manage shows. Authentication is a precondition for protected operations.

3.6 Main booking sequence

1. User opens a movie. 2. Client requests theatre/show information. 3. User selects a show. 4. Client loads seat availability. 5. User selects seats. 6. Client submits booking/payment-related requests as implemented. 7. Server updates the show bookedSeats and creates a booking record. 8. Client displays confirmation. Verify exact payment ordering and failure rollback in the running system.

3.7 API route groups

User routes are mounted at /api/user; movie routes at /api/movie; theatre routes at /api/theatre; booking routes at /api/bookings. Route names visible in source include register, login, get-current-user, get-all-movies, add/update/delete movie, theatre/show management, make-payment, book-show and get-all-bookings.

3.8 Security design

Passwords are hashed with bcryptjs during registration. JWT tokens are created at login and authMiddleware protects selected routes. Review the route-level middleware carefully: some booking endpoints in the supplied code are not protected, so harden authorization before production use.

3.9 Design diagrams to include in final submission

Use the descriptions above to draw architecture, ER, use-case, sequence and DFD Level 0/1 diagrams in draw.io. Do not label a diagram as generated from runtime behavior unless verified against the implementation.

Accuracy note: This document describes the code and README found in the supplied ZIP. Items marked as recommendations or proposed tests are not claims that they have already been implemented or passed. Confirm all workflows on your laptop before submission.